

M07 PROTECTED PRACTICE REVIEW

Open only after a genuine saved notebook attempt

Compare logic/evidence -> repair -> close -> changed retry -> append evidence.

PY0 review - Python, Jupyter and a reproducible analyst notebook

1. Strong attempt should demonstrate

Save shape/key/type/count/total evidence appropriate to the lesson. A cell running without an error is not by itself validation.

2. Expected reasoning / teacher pattern

A notebook mixes explanation and executable code. That is useful for analysis because another person can see not only the result, but how you obtained it. A reproducible notebook should run from the first cell to the last without depending on hidden state. A cell that works only because you ran some forgotten cell earlier is not a reliable analysis. Worked pattern: `import sys`
`print(sys.version)` `import pandas as pd` `import matplotlib` `print(pd.__version__)` `print(matplotlib.__version__)`

3. What to compare

Compare source/result grain, before/after row/key counts and independent validation before comparing exact code text. Equivalent clear pandas/Python code is acceptable.

4. Likely repair

If a variable exists only because an earlier cell was run manually, restart the kernel and run all. If imports fail, print `sys.executable` and package versions before reinstalling anything.

5. Fresh retry

Close this review. Run the changed task from the practice notebook without reopening answer material. Append the new attempt and validation; only the fresh retry upgrades mastery.

PY1 review - Practical Python basics: values, conditions, loops and functions

1. Strong attempt should demonstrate

Save shape/key/type/count/total evidence appropriate to the lesson. A cell running without an error is not by itself validation.

2. Expected reasoning / teacher pattern

Python fundamentals matter only because they let you express business logic. A variable gives a value a name. A condition chooses between outcomes. A loop repeats a small operation. A function packages a rule so you can reuse and test it. For analyst work, clarity is more important than clever code. Worked pattern: revenue = 125000 cost = 82000 profit = revenue - cost
`def sales_band(net_sales): if net_sales >= 100000: return "High" elif net_sales >= 50000: return "Medium" return "Low"`
`print(profit, sales_band(revenue))`

3. What to compare

Compare source/result grain, before/after row/key counts and independent validation before comparing exact code text. Equivalent clear pandas/Python code is acceptable.

4. Likely repair

If a condition fails at a boundary, test values just below, exactly at, and just above the threshold. If a loop is confusing, print the current item and accumulator each iteration.

5. Fresh retry

Close this review. Run the changed task from the practice notebook without reopening answer material. Append the new attempt and validation; only the fresh retry upgrades mastery.

PY2 review - pandas: load, inspect, select and filter

1. Strong attempt should demonstrate

Save shape/key/type/count/total evidence appropriate to the lesson. A cell running without an error is not by itself validation.

2. Expected reasoning / teacher pattern

A pandas DataFrame is a table-like object. Before analysis, inspect the dataset rather than assuming its shape or data types. Useful first checks: `head()`, `shape`, `columns`, `dtypes`, `info()`, missing counts and key uniqueness. Filtering changes which rows are in scope, so filters must reflect the business question. Worked pattern: `import pandas as pd`
`sales = pd.read_csv("Practice_Data/sales.csv")`
`print(sales.head())`
`print(sales.shape)`
`print(sales.dtypes)`
`print(sales.isna().sum())`
`east = sales.loc[sales["region"].eq("East"), ["order_id", "order_date", "product_id", "quantity"]]`

3. What to compare

Compare source/result grain, before/after row/key counts and independent validation before comparing exact code text. Equivalent clear pandas/Python code is acceptable.

4. Likely repair

If filtering returns unexpected rows, inspect dtype and unique values before changing the condition. If a date comparison fails, parse the column explicitly first.

5. Fresh retry

Close this review. Run the changed task from the practice notebook without reopening answer material. Append the new attempt and validation; only the fresh retry upgrades mastery.

PY3 review - Data quality and cleaning in pandas

1. Strong attempt should demonstrate

Save shape/key/type/count/total evidence appropriate to the lesson. A cell running without an error is not by itself validation.

2. Expected reasoning / teacher pattern

Cleaning is not 'make warnings disappear.' Preserve evidence, identify the rule, investigate impact and document the decision. Use `.str.strip()` for whitespace, `pd.to_datetime(..., errors='coerce')` when you want invalid date text to become visible as missing, `duplicated()` for duplicate detection, and explicit range/rule checks for invalid values. Worked pattern: `work = sales.copy()`
`work["customer_id"] = work["customer_id"].astype("string").str.strip()` `work["region"] =`
`work["region"].astype("string").str.strip().str.title()` `work["order_date"] = pd.to_datetime(work["order_date"], errors="coerce")`
`duplicate_lines = work[work.duplicated("order_line_id", keep=False)]` `bad_discount = work.loc[~work["discount"].between(0,1)]`

3. What to compare

Compare source/result grain, before/after row/key counts and independent validation before comparing exact code text. Equivalent clear pandas/Python code is acceptable.

4. Likely repair

If cleaning reduces row count unexpectedly, compare before/after counts and isolate the exact rule. Never use `dropna/drop_duplicates` until you can name the business reason.

5. Fresh retry

Close this review. Run the changed task from the practice notebook without reopening answer material. Append the new attempt and validation; only the fresh retry upgrades mastery.

PY4 review - GroupBy, aggregation and result grain

1. Strong attempt should demonstrate

Save shape/key/type/count/total evidence appropriate to the lesson. A cell running without an error is not by itself validation.

2. Expected reasoning / teacher pattern

groupby changes grain. If you group by region, one output row represents one region. If you group by month and category, each output row represents one month-category combination. Always check whether a metric is additive at that grain. Counts, distinct counts and averages answer different questions. Worked pattern: `summary = ( work.groupby("region", dropna=False) .agg( rows=("order_line_id","size"), orders=("order_id","nunique"), net_sales=("net_sales","sum") ) .reset_index() )`

3. What to compare

Compare source/result grain, before/after row/key counts and independent validation before comparing exact code text. Equivalent clear pandas/Python code is acceptable.

4. Likely repair

If totals look wrong, state input grain and output grain, then reconcile the grouped sum to the ungrouped total. Distinguish row count from nunique business entities.

5. Fresh retry

Close this review. Run the changed task from the practice notebook without reopening answer material. Append the new attempt and validation; only the fresh retry upgrades mastery.

PY5 review - Merge, concat and validation

1. Strong attempt should demonstrate

Save shape/key/type/count/total evidence appropriate to the lesson. A cell running without an error is not by itself validation.

2. Expected reasoning / teacher pattern

A merge is the Python equivalent of a relational join. It can silently multiply rows if the right-side key is not unique. Before a many-to-one enrichment merge, test the dimension key for duplicates. During merge, `validate='many\_to\_one'` makes pandas raise an error if the assumed relationship is false. `indicator=True` exposes unmatched rows. Worked pattern: `products = pd.read_csv("Practice_Data/products.csv") assert not products["product_id"].duplicated().any() merged = work.merge( products, on="product_id", how="left", validate="many_to_one", indicator=True ) print(merged["_merge"].value_counts()) print(len(work), len(merged))`

3. What to compare

Compare source/result grain, before/after row/key counts and independent validation before comparing exact code text. Equivalent clear pandas/Python code is acceptable.

4. Likely repair

If merge validation fails, stop and inspect duplicate keys on each side. Do not remove `validate=` to make the error disappear.

5. Fresh retry

Close this review. Run the changed task from the practice notebook without reopening answer material. Append the new attempt and validation; only the fresh retry upgrades mastery.

PY6 review - Reshape and time analysis

1. Strong attempt should demonstrate

Save shape/key/type/count/total evidence appropriate to the lesson. A cell running without an error is not by itself validation.

2. Expected reasoning / teacher pattern

Analyst data often moves between long and wide forms. Long form is usually easier for filtering, grouping and plotting. A pivot table is useful for a matrix view such as Month x Region. Time analysis begins with real datetime values, not strings that merely look like dates. Worked pattern: `merged["order_date"] = pd.to_datetime(merged["order_date"], errors="coerce")` `merged["month"] = merged["order_date"].dt.to_period("M").astype(str)` `monthly = merged.groupby(["month", "region"], as_index=False)["net_sales"].sum()` `wide = monthly.pivot(index="month", columns="region", values="net_sales")` `long_again = wide.reset_index().melt(id_vars="month", var_name="region", value_name="net_sales")`

3. What to compare

Compare source/result grain, before/after row/key counts and independent validation before comparing exact code text. Equivalent clear pandas/Python code is acceptable.

4. Likely repair

If pivot/melt totals change, compare total before reshape and after returning to long form. Check duplicate index/column combinations.

5. Fresh retry

Close this review. Run the changed task from the practice notebook without reopening answer material. Append the new attempt and validation; only the fresh retry upgrades mastery.

PY7 review - Visualization with pandas and Matplotlib

1. Strong attempt should demonstrate

Save shape/key/type/count/total evidence appropriate to the lesson. A cell running without an error is not by itself validation.

2. Expected reasoning / teacher pattern

Use a line chart for ordered time trends, a bar chart for category comparison and a histogram/boxplot for distribution questions. A chart is part of the argument, not decoration. The goal is not to memorize every Matplotlib option. Create a clear first chart, then improve labels and formatting only as needed. Worked pattern: `import matplotlib.pyplot as plt trend = merged.groupby("month", as_index=False)["net_sales"].sum() ax = trend.plot(x="month", y="net_sales", kind="line", marker="o", legend=False)`
`ax.set_title("Monthly Net Sales") ax.set_xlabel("Month") ax.set_ylabel("Net Sales") plt.xticks(rotation=45) plt.tight_layout()`
`plt.show()`

3. What to compare

Compare source/result grain, before/after row/key counts and independent validation before comparing exact code text. Equivalent clear pandas/Python code is acceptable.

4. Likely repair

If the chart looks wrong, inspect the summarized data feeding the chart before formatting. Chart polish cannot repair wrong aggregation.

5. Fresh retry

Close this review. Run the changed task from the practice notebook without reopening answer material. Append the new attempt and validation; only the fresh retry upgrades mastery.

PY8 review - Exploratory data analysis (EDA)

1. Strong attempt should demonstrate

Save shape/key/type/count/total evidence appropriate to the lesson. A cell running without an error is not by itself validation.

2. Expected reasoning / teacher pattern

EDA is not 'make every chart.' Start with structure and quality, then examine distributions, categories, trends, grouped metrics and unusual values that matter to the question. An observation becomes useful when you connect it to context: what changed, where, how much, compared with what, and what limitation remains. Worked pattern: `print(merged.shape)` `print(merged.info())` `print(merged.describe(include="all"))` `print(merged.isna().sum())` # Then: category counts, grouped metrics, trends, # selected plots, findings and limitations.

3. What to compare

Compare source/result grain, before/after row/key counts and independent validation before comparing exact code text. Equivalent clear pandas/Python code is acceptable.

4. Likely repair

If EDA becomes a chart dump, return to the business question and keep only checks/visuals that reveal structure, quality, comparison or trend.

5. Fresh retry

Close this review. Run the changed task from the practice notebook without reopening answer material. Append the new attempt and validation; only the fresh retry upgrades mastery.

PY9 review - Notebook quality, outputs and reproducibility

1. Strong attempt should demonstrate

Save shape/key/type/count/total evidence appropriate to the lesson. A cell running without an error is not by itself validation.

2. Expected reasoning / teacher pattern

A professional notebook should have a visible question, imports/setup, loading, quality checks, preparation, analysis, validation, visuals, findings and limitations. Do not scatter unexplained code across random cells. Avoid writing over raw data. Save clean outputs with clear filenames. Before delivery: Restart Kernel -> Run All -> confirm no error. Worked pattern: `from pathlib import Path`
`OUTPUT = Path("outputs")`
`OUTPUT.mkdir(exist_ok=True)`
`analysis_table.to_csv(OUTPUT / "analysis_table.csv", index=False)`

3. What to compare

Compare source/result grain, before/after row/key counts and independent validation before comparing exact code text. Equivalent clear pandas/Python code is acceptable.

4. Likely repair

If Restart + Run All fails, the notebook has hidden state, missing setup, or path assumptions. Fix the earliest failing cell, not the last visible symptom.

5. Fresh retry

Close this review. Run the changed task from the practice notebook without reopening answer material. Append the new attempt and validation; only the fresh retry upgrades mastery.

PY10 review - Integrated analyst workflow in Python

1. Strong attempt should demonstrate

Save shape/key/type/count/total evidence appropriate to the lesson. A cell running without an error is not by itself validation.

2. Expected reasoning / teacher pattern

The final Python skill is not a new function. It is integration. You receive an unfamiliar request and unfamiliar data, then decide what to check, what to calculate, how to validate and how to communicate. Your notebook is evidence of reasoning. A complicated notebook with weak validation is worse than a simple notebook that answers the business question correctly.

Worked pattern: # Final workflow # 1. Frame question/metrics # 2. Load/inspect # 3. Quality audit # 4. Clean/prepare # 5. Merge + validate # 6. Analyze/group/reshape # 7. Visualize # 8. Validate results # 9. Findings/recommendation/limitations # 10. Restart + Run All + save outputs

3. What to compare

Compare source/result grain, before/after row/key counts and independent validation before comparing exact code text. Equivalent clear pandas/Python code is acceptable.

4. Likely repair

If the project becomes too complicated, reduce it to one question, a small metric set, explicit validation, and evidence-based findings. More code is not a better analysis.

5. Fresh retry

Close this review. Run the changed task from the practice notebook without reopening answer material. Append the new attempt and validation; only the fresh retry upgrades mastery.

M-PY project review - grain before code

1. Expected architecture

Question -> raw frames -> issue log -> strict analysis set -> validated product/customer enrichment -> grouped summaries -> order-grain sales + returns -> figures -> reconciliations -> findings/limitations -> outputs -> clean rerun.

2. Protected anchors

Practice fixture: raw sales rows 160; duplicate-ID rows 2; strict analysis rows 157; net sales 1,967,004.30; refund total 229,199.20; net after refunds 1,737,805.10. Use these only to diagnose a saved project attempt.

3. Likely failure routing

Cleaning/row rules -> PY3. Group grain -> PY4. Merge/cardinality -> PY5. Date/reshape -> PY6. Visual -> PY7. EDA interpretation -> PY8. Reproducibility/output -> PY9. Integration/over-complexity -> PY10.

4. Fresh retry

Close this review. Change one business question/scope and rebuild the affected workflow from raw data. Rerun downstream validations and Restart + Run All before Gate A.